Что я изучил: существующие ИИ-агенты для граничных устройств

Студент	Евстигнеев Н. А., группа P3431
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.; данные о проектах (звёзды на GitHub, версии) — на 6 октября 2026 г.

> **Итог обзора в трёх пунктах.** (1) Схема «тонкий агент на устройстве + внешняя языковая модель + MCP к оборудованию» уже стала распространённой практикой, поэтому новизна работы не в самой схеме. (2) Существующие агенты оценивались лишь на коротких задачах; многочасового профиля «вызовы модели + токены + рубли + память + процессор» на одноплатнике, а тем более при жёстких ограничениях ресурсов, в проверенных источниках нет. (3) Для робота и камеры главная статья затрат — сырые данные датчиков: их нельзя отдавать модели напрямую.

1. Классы существующих агентов

Класс	Примеры	Где языковая модель	Что важно для ВКР
Личные агенты на собственном оборудовании пользователя и их облегчённые варианты	OpenClaw; NanoClaw, PicoClaw, nanobot, ZeroClaw; агенты на микроконтроллерах ESP32	облако	ближайшие аналоги и основа сравнения
Умный дом и голосовые ассистенты	Home Assistant с языковой моделью и MCP-сервером; xiaozhi-esp32	облако или локально	массовая практика «оборудование → MCP → модель», часто на Raspberry Pi
Робототехника (ROS)	ROSA (NASA JPL), ros-mcp-server, Nav2-MCP; агентные «ОС» роботов	облако или машина оператора	сценарий с роботом; пока в основном исследования, а не массовое использование
Экосистема NVIDIA Jetson	jetson-containers, руководства по запуску OpenClaw на Jetson Orin, jetson-inference	на борту (Orin)	для старого Jetson Nano применимо мало: локальные модели на нём медленные
Универсальные фреймворки	LangChain/LangGraph, OpenAI Agents SDK, smolagents и др.	любая	тяжёлые для слабых плат; в лёгких агентах вместо них реализован собственный минимальный цикл
Промышленный IoT	AWS Greengrass + облачный агент, решения Microsoft и Siemens	облако или локальный сервер	схема «дешёвый локальный фильтр → облачный агент»

Научные системы	SayCan, Code as Policies, ChatGPT for Robotics, Sasha, SimuHome, IoT-MCP	в основном облако	методы оценки, метрики, обоснование

2. Ближайшие аналоги: что они собой представляют и сколько потребляют

Система	Язык и запуск	Память (сторонние замеры; где указано — наш)	Старый Jetson Nano (Ubuntu 18.04)
OpenClaw (≈391 тыс. звёзд)	TypeScript, Node.js ⩾ 24.16	медиана пиковой памяти ≈2,9 ГиБ (Docker на рабочей станции) [1]	штатно не запускается: сборки Node.js требуют более новой системы
NanoClaw (≈31 тыс.)	TypeScript, Node.js ⩾ 22, Docker	—	штатно не запускается
PicoClaw (≈30 тыс.)	Go, один исполняемый файл	≈31 МБ пиковой памяти на Raspberry Pi 4 [2]; в нашем пробном прогоне (v0.3.1, ноутбук) — ≈42 МиБ	должен запускаться (статическая сборка)
nanobot (≈49 тыс.)	Python	≈130 МБ на Raspberry Pi 4 [2]; ≈136 МиБ (версия 0.1.5) [1]; в нашем пробном прогоне (версия 0.3.5) — ≈166 МиБ	только с ручной сборкой одной зависимости
ZeroClaw	Rust	≈20 МБ [2]	сборка musl статическая — должна запускаться (сборка gnu — нет)
ros-mcp-server	Python, MCP-мост к роботу через rosbridge	≈110 МиБ (собственный замер)	—

Какие приёмы экономии уже есть у аналогов: фильтр «будить ли модель» (NanoClaw), правила, которые модель создаёт один раз и которые затем исполняются без неё, событийные триггеры вместо опроса, облегчённый контекст фоновой проверки, лимиты вызовов, отложенная загрузка описаний инструментов. Эти приёмы в работе не выдаются за новые; измеряется, когда и насколько каждый из них окупается. Чего у PicoClaw и nanobot нет (проверено по их коду): их MCP-клиенты берут у сервера только инструменты и не подписываются на изменения ресурсов, поэтому события датчиков до них сами не доходят — на стенде их доставляет внешний фильтр.

3. Почему языковая модель внешняя (как и решили на встрече)

- На одноплатнике с несколькими гигабайтами памяти практически пригодны модели примерно до 1,5 млрд параметров, но и они медленные: на Raspberry Pi 5 — 5–15 токенов в секунду [3]; на оригинальном Jetson Nano модели около 1 млрд параметров дают 4–7 токенов в секунду (любительские замеры сообщества).
- В независимом замере на Raspberry Pi 4 ни один из агентов не справился с задачами на локальных малых моделях (Qwen2.5-1.5B, SmolLM2-360M) [2].
- В реальных умных домах облачные модели выбирают заметно чаще локальных: по данным открытой аналитики Home Assistant, интеграцию Gemini используют около

34,8 тыс. установок, OpenAI — 24,5 тыс., локальную Ollama — 12,6 тыс.

4. Протокол MCP и «контекст оборудования»

MCP (Model Context Protocol) — открытый протокол, по которому агент получает от серверов **инструменты** (вызвать функцию: прочитать датчик, включить реле, поехать к точке) и **ресурсы** (данные, на изменения которых можно подписаться). В 2025–2026 гг. он стал стандартом де-факто для доступа агентов к оборудованию: MCP-сервер Home Assistant (около 25,8 тыс. установок), ros-mcp-server для роботов, xiaozhi-esp32 для микроконтроллеров [4, 5]. Практика подтверждает топологию, выбранную в работе: на стороне оборудования работает MCP-сервер, агент выступает клиентом.

5. Что я взял из научных работ

- **Определения** граничного ИИ и агента [6, 7]; многоуровневая архитектура, в которой нижние уровни работают без «рассуждающего» модуля [8].
- **Оценка агентов по затратам**, а не только по точности, с обязательным простым базовым вариантом [9].
- **Реалистичная нагрузка дня**: частота команд голосовым ассистентам из многомесячного исследования 88 домов (в среднем 2,6; 5,6 и 30 команд в сутки у разных групп пользователей) [10] и записи датчиков реальных домов CASAS [11].
- **Сокращение истории**: простое скрытие старых результатов инструментов не хуже составления сводки моделью и дешевле [12].
- **Безопасность приводов**: правила, записанные в запросе, модель соблюдает не полностью, поэтому нужен отдельный детерминированный модуль проверки [13].
- **MCP для датчиков** на граничных узлах: среднее время ответа 205 мс, 12-часовой прогон, но без учёта токенов и стоимости [5].
- **Бенчмарки умного дома** для формулировки задач и проверки по итоговому состоянию устройств [14, 15].

6. Пробел, который закрывает работа

Существующие решения либо ориентированы на функциональность и считают в лучшем случае токены и стоимость отдельных запросов (например, /usage cost в OpenClaw), но не профиль затрат и ресурсов платы за многочасовую работу (OpenClaw, PicoClaw, nanobot), либо дают доступ к оборудованию без агента и без метрик (ros-mcp-server, MCP-сервер Home Assistant). Независимые измерения агентов этого класса на одноплатниках ограничены отдельными задачами и не охватывают ни многочасовую работу, ни предельно слабое оборудование. Поэтому в работе разрабатывается агент с MCP-доступом к оборудованию и встроенным учётом ресурсов и затрат, а также методика оценки его работы во времени, включая сравнение с аналогами на одной и той же нагрузке.

Источники (основные)

1. Salman A. Resource Constraints and Performance in Agentic AI Systems / A. Salman, M. Halgamuge, T. Susnjak. – 2026. – arXiv:2608.27886 (препринт).

2. Dehigaspitiyage Don S. Y. Benchmarking Lightweight AI-Agent Frameworks on a 2 GB Raspberry Pi 4: Framework Overhead, Offline Feasibility, and Fleet Density. – Zenodo, 2026. – DOI 10.5281/zenodo.20593332 (препринт).
3. Nguyen T. An Evaluation of LLMs Inference on Popular Single-board Computers / T. Nguyen, T. Nguyen. – 2025. – arXiv:2511.07425 (препринт).
4. Model Context Protocol : specification, version 2025-11-25. – URL: https://modelcontextprotocol.io/specification/2025-11-25 (дата обращения: 06.10.2026).
5. IoT-MCP: Bridging LLMs and IoT Systems Through Model Context Protocol / N. Yang, G. Lyu, M. Ma [et al.] // Proceedings of WiNTECH '25. – 2025. – P. 73–80. – DOI 10.1145/3737895.3768303.
6. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing / Z. Zhou, X. Chen, E. Li [et al.] // Proceedings of the IEEE. – 2019. – Vol. 107, № 8. – P. 1738–1762. – DOI 10.1109/JPROC.2019.2918951.
7. Wooldridge M. Intelligent agents: theory and practice / M. Wooldridge, N. R. Jennings // The Knowledge Engineering Review. – 1995. – Vol. 10, № 2. – P. 115–152. – DOI 10.1017/S0269888900008122.
8. Brooks R. A. A robust layered control system for a mobile robot // IEEE Journal on Robotics and Automation. – 1986. – Vol. 2, № 1. – P. 14–23. – DOI 10.1109/JRA.1986.1087032.
9. AI Agents That Matter / S. Kapoor, B. Stroebl, Z. S. Siegel [et al.] // Transactions on Machine Learning Research. – 2025. – URL: https://openreview.net/forum?id=Zy4uFzMviZ.
10. Understanding the Long-Term Use of Smart Speaker Assistants / F. Bentley, C. Luvogt, M. Silverman [et al.] // Proceedings of the ACM on IMWUT. – 2018. – Vol. 2, № 3. – Art. 91. – DOI 10.1145/3264901.
11. CASAS: A Smart Home in a Box / D. J. Cook, A. S. Crandall, B. L. Thomas, N. C. Krishnan // Computer. – 2013. – Vol. 46, № 7. – P. 62–69. – DOI 10.1109/MC.2012.328.
12. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management / T. Lindenbauer, I. Slinko, L. Felder [et al.] // DL4C Workshop at NeurIPS 2025. – 2025. – URL: https://openreview.net/forum?id=OHVzruJl5k.
13. Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents / Z. Yang, S. S. Raman, A. Shah, S. Tellex // ICRA 2024. – 2024. – P. 14435–14442. – DOI 10.1109/ICRA57147.2024.10611447.
14. SimuHome: A Temporal- and Environment-Aware Benchmark for Smart Home LLM Agents / G. Seo, J. Yang, J. Pyo [et al.] // ICLR 2026. – 2026. – URL: https://openreview.net/forum?id=LCS1WsGvha.
15. HomeBench: Evaluating LLMs in Smart Homes with Valid and Invalid Instructions Across Single and Multiple Devices / S. Li, Y. Guo, J. Yao [et al.] // Proceedings of ACL 2025 (Volume 1: Long Papers). – 2025. – P. 12230–12250. – DOI 10.18653/v1/2025.acl-long.597.

Полная карта агентов с проверкой фактов — `research/new_topic/landscape/AGENT_LANDSCAPE.md`; полный список проверенных источников — `research/new_topic/SOURCES.md`.